

NILS HARTMANN

https://nilshartmann.net

Das Backend im Frontend Next.js

W-JAX MÜNCHEN | 5. NOVEMBER 2026 | HTTPS://REACT.SCHULE

NILS HARTMANN

nils@nilshartmann.net

Freiberuflicher Software-Entwickler, –Architekt, Coach, Trainer

Java, Spring, GraphQL, React, TypeScript



https://react.schule/video-kurs-react



https://reactbuch.de

NEXT.JS

https://nextjs.org

https://nextjs.org

- basiert auf React

https://nextjs.org

- basiert auf React
- fügt serverseitiges Rendern, Caching und Routing hinzu

NEXT.JS

https://nextjs.org

- basiert auf React
- fügt serverseitiges Rendern, Caching und Routing hinzu
- ähnlich wie Spring Boot, das z.B. Webserver + JSP/Thymeleaf integriert

https://nextjs.org

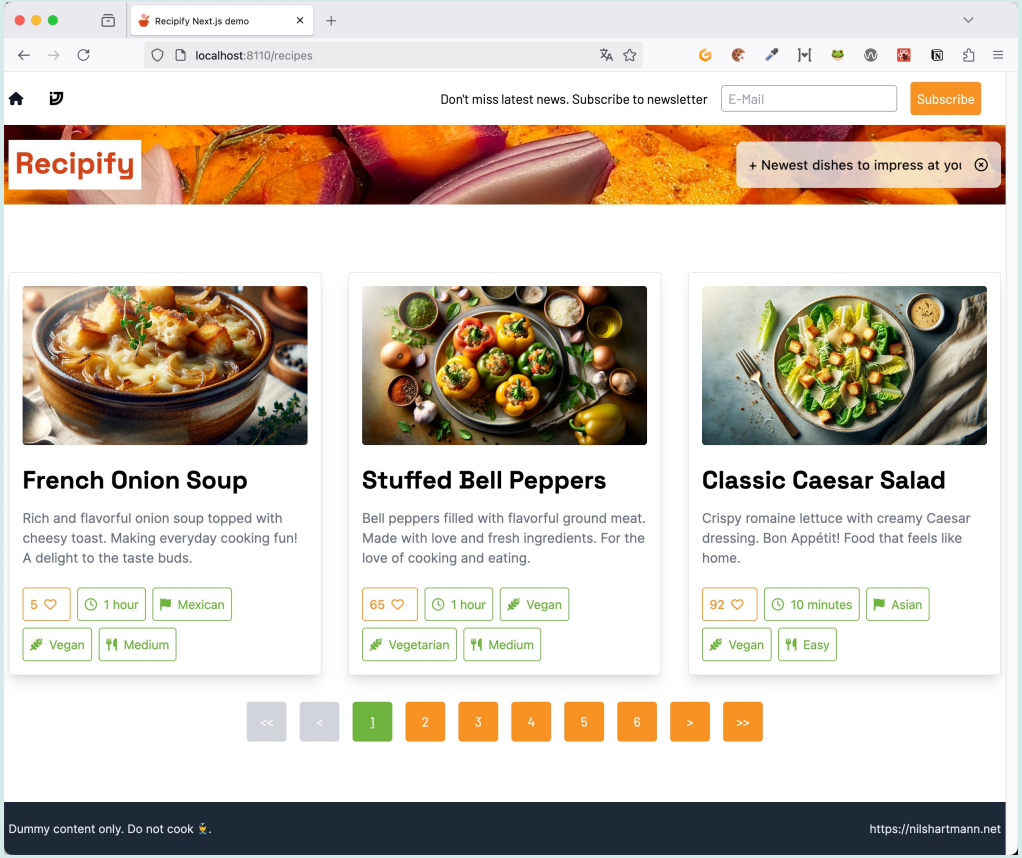
- basiert auf React
- fügt serverseitiges Rendern, Caching und Routing hinzu
- ähnlich wie Spring Boot, das z.B. Webserver + JSP/Thymeleaf integriert

- Ziel:
  - Hohe Performance (oder die Illusion davon)

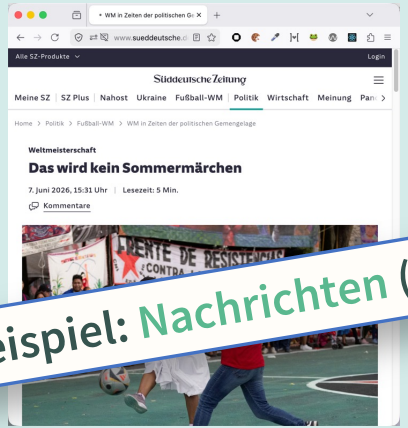
EIN BEISPIEL...

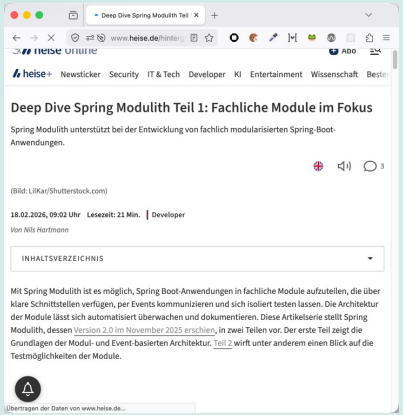


EIN BEISPIEL...



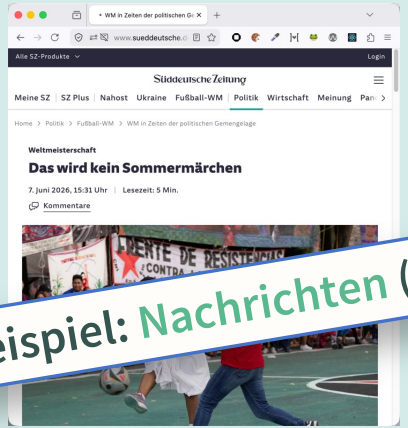
UNTERSCHIEDLICHE WEBANWENDUNGEN



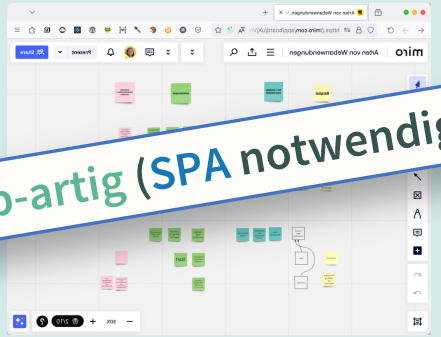


Beispiel: Nachrichten (eher statisch)

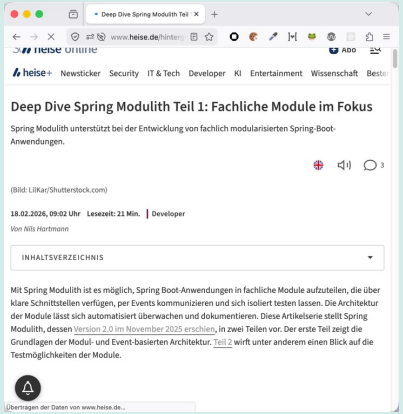
UNTERSCHIEDLICHE WEBANWENDUNGEN

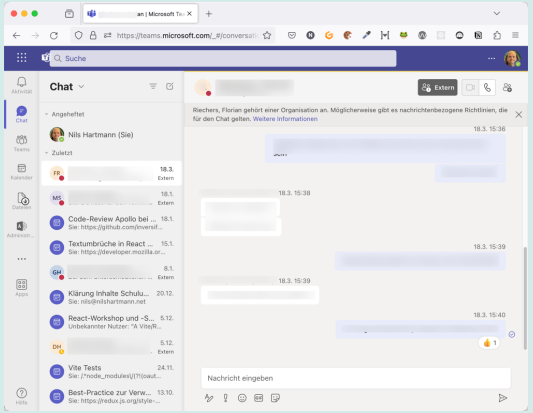


Beispiel: Nachrichten (eher statisch)

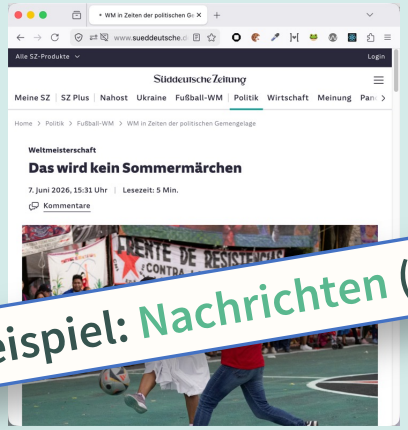


Beispiel: Desktop-artig (SPA notwendig)

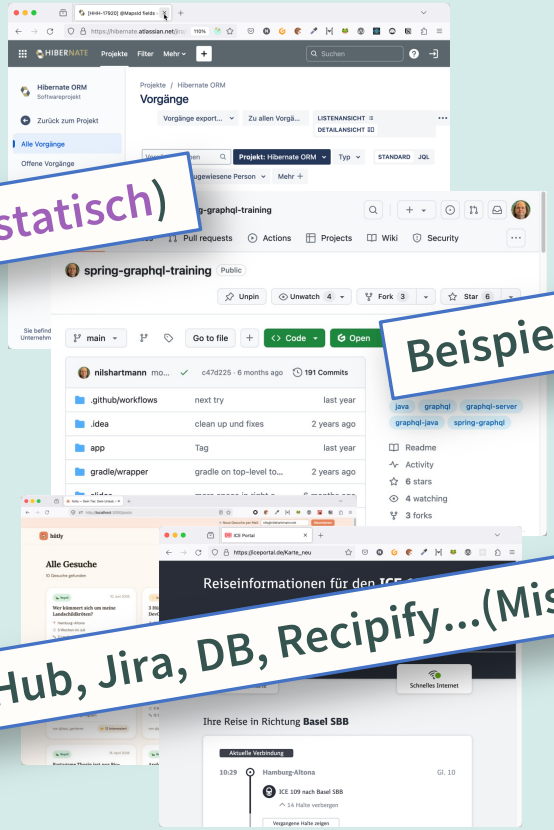




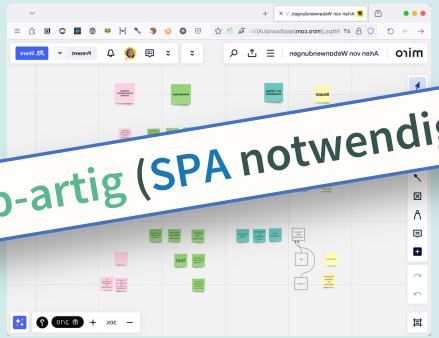
UNTERSCHIEDLICHE WEBANWENDUNGEN



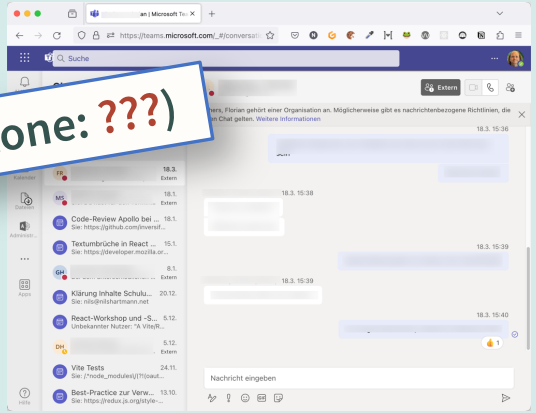
Beispiel: Nachrichten (eher statisch)



Beispiel: GitHub, Jira, DB, Recipify...(Mischzone: ???)



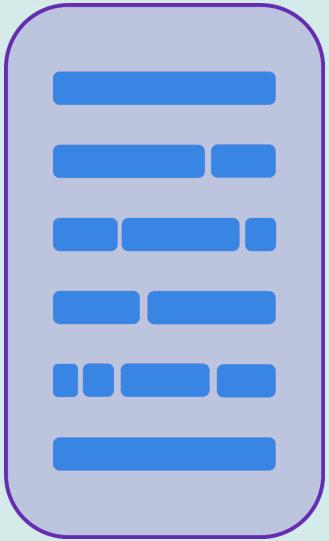
Beispiel: Desktop-artig (SPA notwendig)



UNTERSCHIEDLICHE WEBANWENDUNGEN



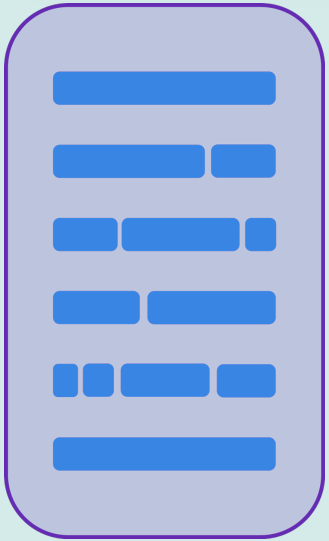
Single-Page-Anwendungen: Stärken...



JavaScript
im
Browser

- UI/UX: flüssige Interaktionen, „App-Gefühl“ 👍
- Architektur-Risiko: 0 (jeder PO-Wunsch abbildbar) 👍
- Aber der **Preis**... 😯

Single-Page-Anwendungen: Preis



JavaScript
im
Browser

- Sehr viel **JavaScript zur Laufzeit**
- Das JavaScript muss laufen, bevor etwas dargestellt wird:
  1. JS laden 🥺
  2. JS ausführen 🥱
  3. Daten laden 😴
  4. Anwendung darstellen zzZ
- JavaScript wird mit jedem Feature mehr 😓
- SEO? Caching? 🫠

Single-Page-Anwendungen: Preis

Können wir am Preis drehen?



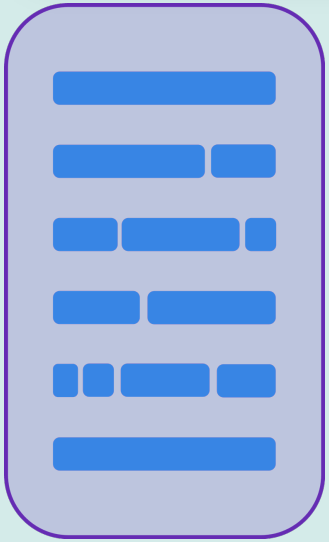
JavaScript
im
Browser

muss laufen, bevor etwas dargestellt wird:

1. JS laden 😩
2. JS ausführen 🥱
3. Daten laden 😴
4. Anwendung darstellen 💤

- JavaScript wird mit jedem Feature mehr 😓
- SEO? Caching? 🫠

Preis senken: ServerSide Rendering (SSR) mit SPAs



- SPA wird nun auch auf dem Server ausgeführt:
  1. Beim initialen Request wird HTML erzeugt ✅
  2. Browser kann HTML sofort darstellen ✅
  3. Danach wird der komplette JS-Code geladen
  4. JavaScript-Code wird ausgeführt („hydration“)
  5. Jetzt ist die Anwendung interaktiv 💪
  6. Weitere Interaktionen rein client-seitig (schnell!) ✅

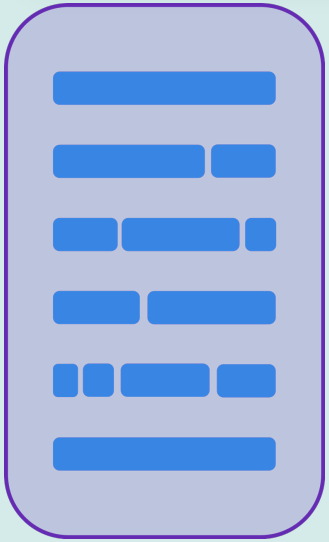
Preis senken: ServerSide Rendering (SSR) mit SPAs



- SPA wird nun auch auf dem S… erzeugt ✅
2. Browser kann HTML sofort darstellen ✅
3. **Danach wird der komplette JS-Code geladen**
4. **JavaScript-Code wird ausgeführt („hydration“)**
5. Jetzt ist die Anwendung interaktiv 💪
6. Weitere Interaktionen rein client-seitig (schnell!) ✅

👀 **JavaScript auch für statische Teile?** 🫣

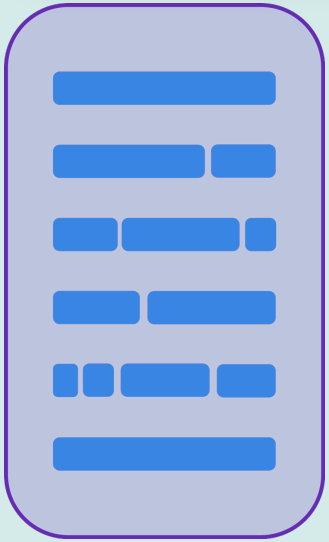
Grenzen von SSR



Überall
JavaScript

- Kompletter JS-Code weiterhin im Browser
  1. Interaktion erst, wenn der komplett ausgeführt ist
  2. Hydration kostet CPU/Akku (auch für statische Teile)
  3. Problem wächst potentiell mit jedem Feature

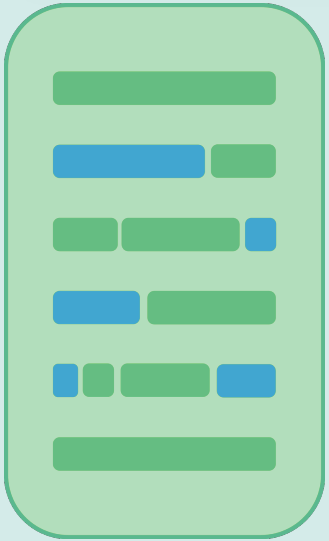
Grenzen von SSR



Überall JavaScript

- Kompletter JS-Code weiterhin im Browser
  1. Interaktion erst, wenn der komplett ausgeführt ist
  2. Hydration kostet CPU/Akku (auch für statische Teile)
  3. Problem wächst potentiell mit jedem Feature
- Code muss auf Server **und** Client lauffähig sein
  1. Kein Zugriff auf Server-Ressourcen (z.B. DB, FS)
  2. API-Keys 😱

Fullstack React mit Next.s: Bestes aus beiden Welten?



im Browser:
HTML+**JS**

- „Fullstack“-Versprechen:
  - Komplette Anwendung mit TypeScript/Next.js **entwickelt**
  - JavaScript **im Browser** nur, wo für Interaktionen

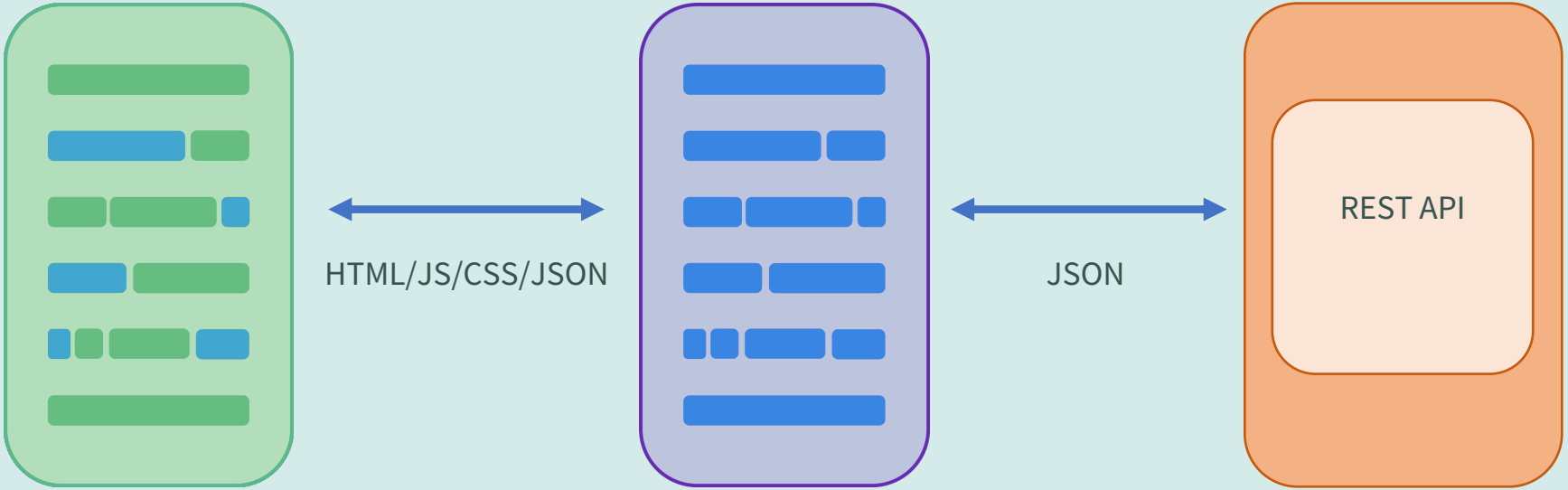
Live

Demo



HTTP://LOCALHOST:8200

Architektur



NILS HARTMANN
https://nilshartmann.net

Vielen Dank!

Source & Code: https://react.schule/wjax-2026-next

Fragen und Kontakt:

nils@nilshartmann.net

https://nilshartmann.net/kontakt